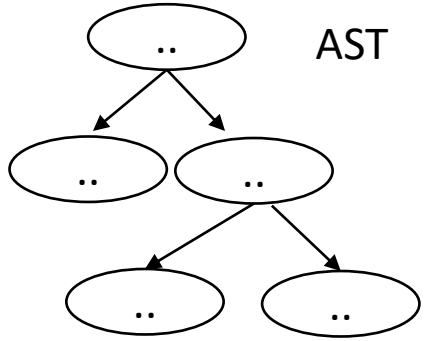
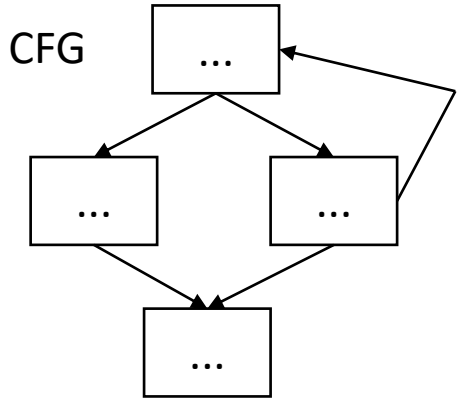
CSE110A: Compilers



MIDTERM REVIEW



PostOrder Traversal Example

3 address code

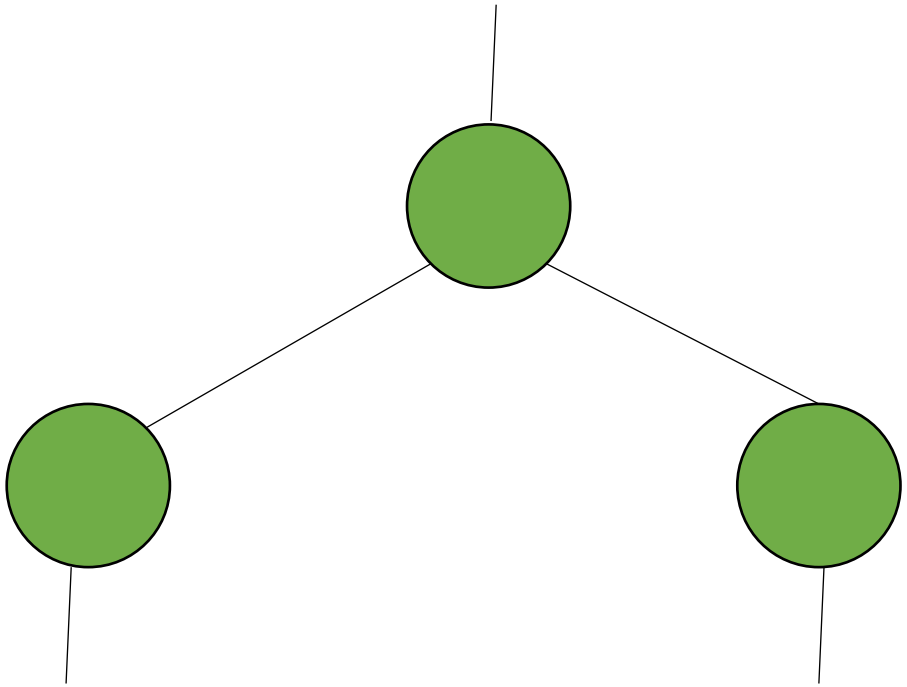
```
store i32 0, ptr %2
%3 = load i32, ptr %1
%4 = add nsw i32 %3, 1,
store i32 %4, ptr %1
%5 = load i32, ptr %2
```

Post-Order Traversal == Postfix

When compilers parse arithmetic or logical expressions, they often build a **parse tree** or **abstract syntax tree (AST)** representing the expression's structure.

Post order traversal of trees naturally matches how expressions can be **evaluated using a stack-based approach** — like in a **stack machine** or when generating **Reverse Polish Notation (RPN)**.

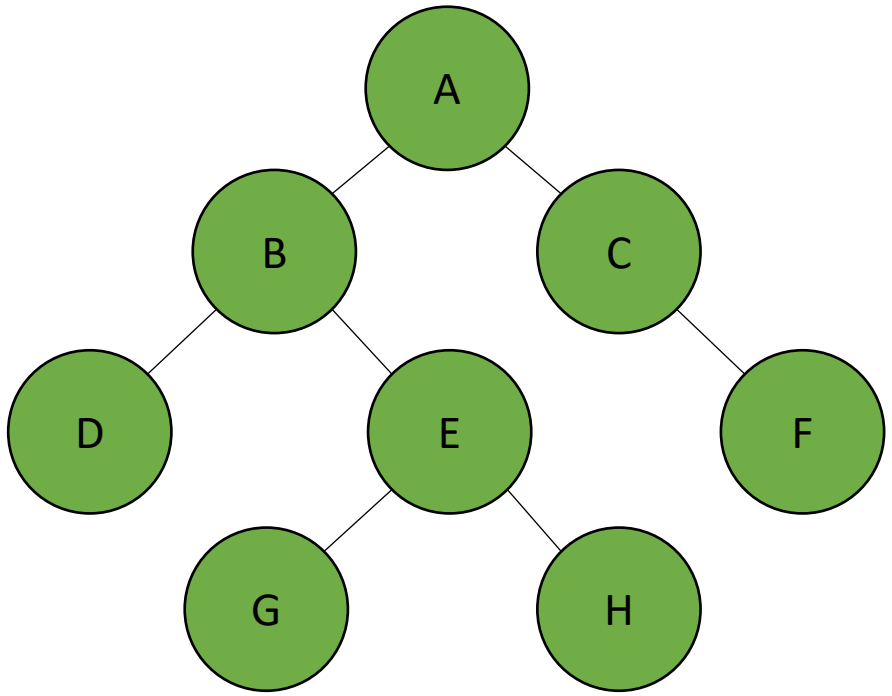
An Aside: Tree Traversal Orders



Let’s review different type of tree traversal algorithms:

pre-order?
in-order?
post-order?

Possible Tree Traversal Orders



Traversal Order	Order Visited	Example Output
pre-order	Top-Root->Left-Child->Right-Child	A B D E G H C F
in-order	Left/Bottom->Its-Root->Right/Bottom Caveat: for binary trees only, not used in compilers	D B G E H A C F
post-order	Left/Bottom->Right/Bottom->Its-Root	D G H E B F C A

Traversals never visit the same node twice.

Pre-order Traversal (Root-Left-Right. Top to Bottom)

Prioritizes visits from top to bottom, and left to right
Useful for serializing and copying trees.

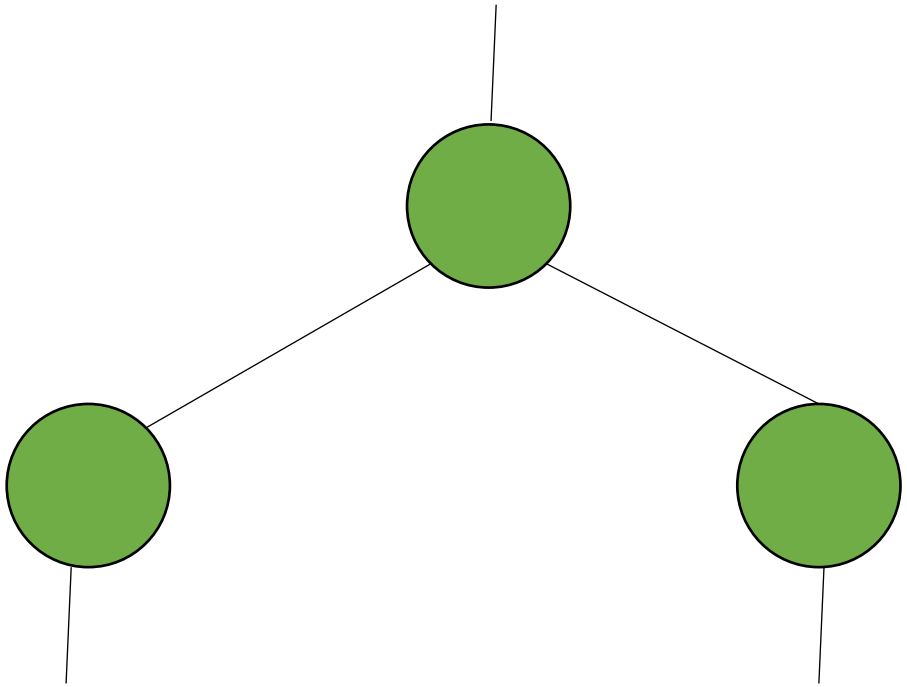
In-order Traversal (Left-Root-Right, Bottom to Top)

Prioritizes visits from bottom left to right visiting parent nodes on the way. Sometimes used for sorting.

Post-order (or natural) Traversal (Left-Right-Root)

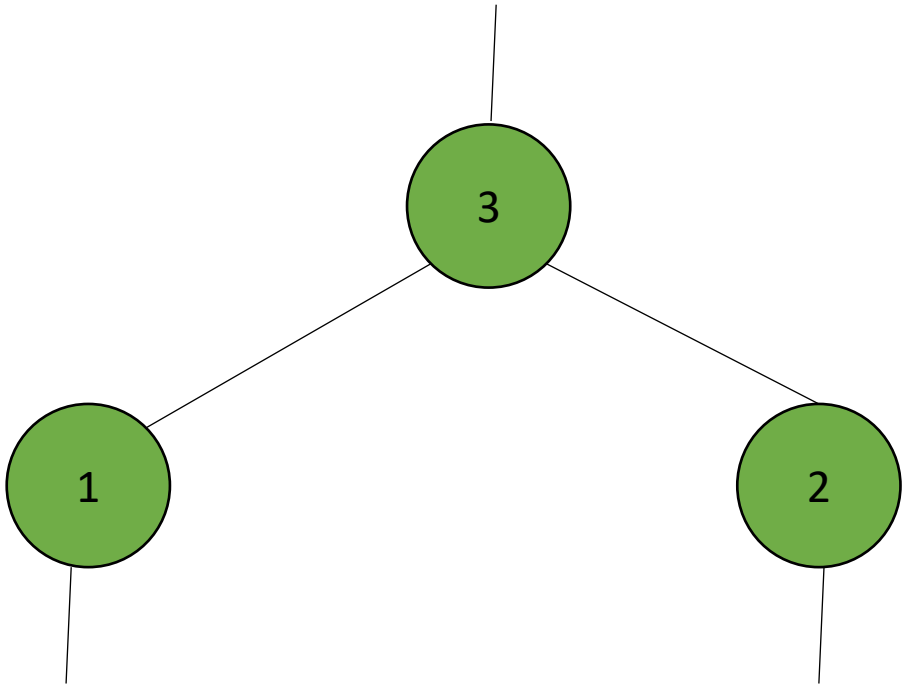
Prioritizes traversing subtrees by visiting lowest nodes first, and parent nodes later. Useful for evaluating expression trees (e.g. parsing)

Post-Order Traversal



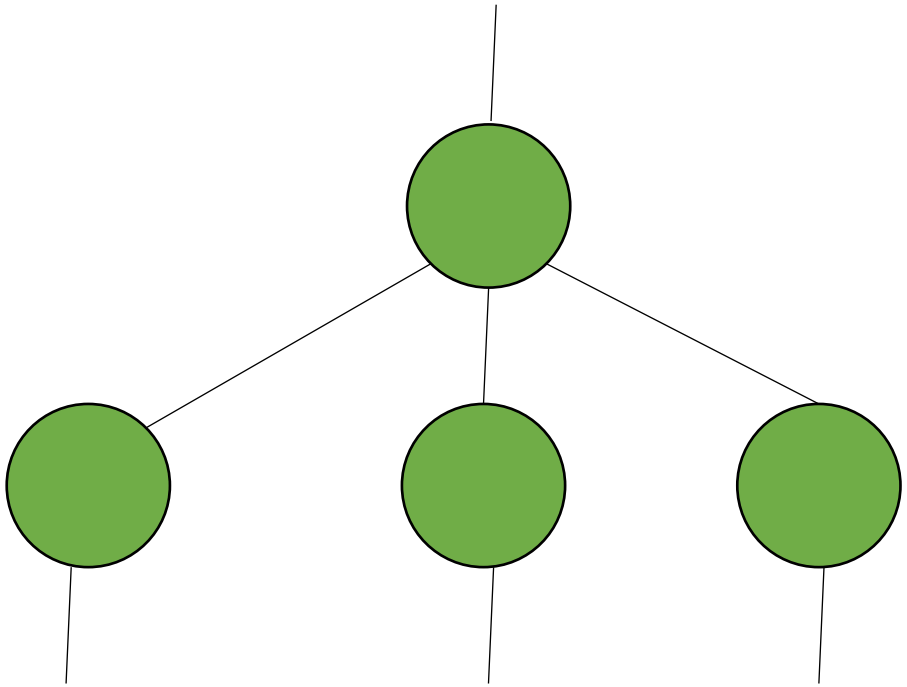
Post order traversal
For 2 children nodes

Post order traversal



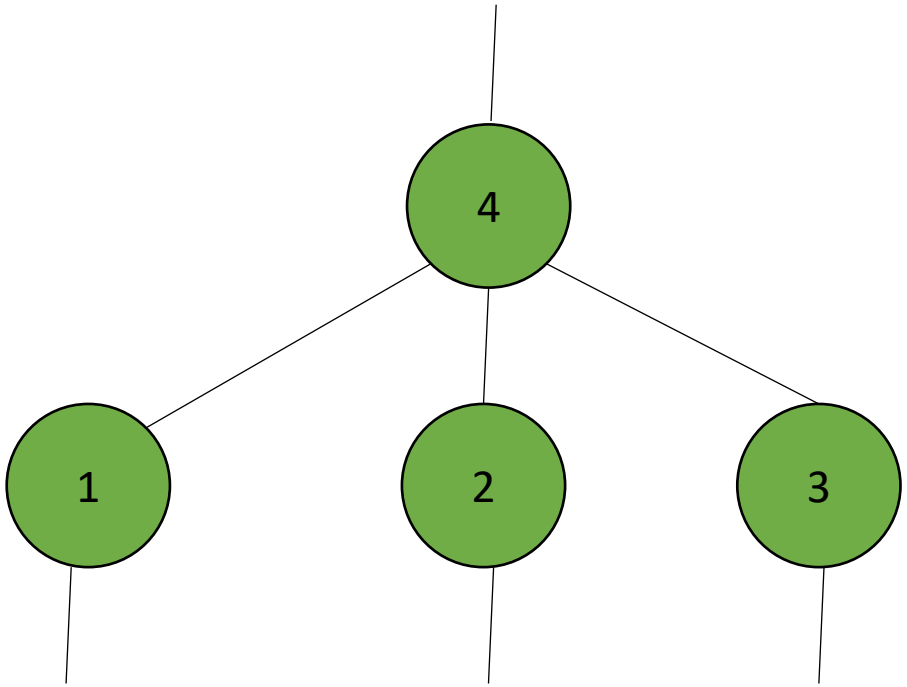
Post order traversal
For 2 children nodes

Post order traversal



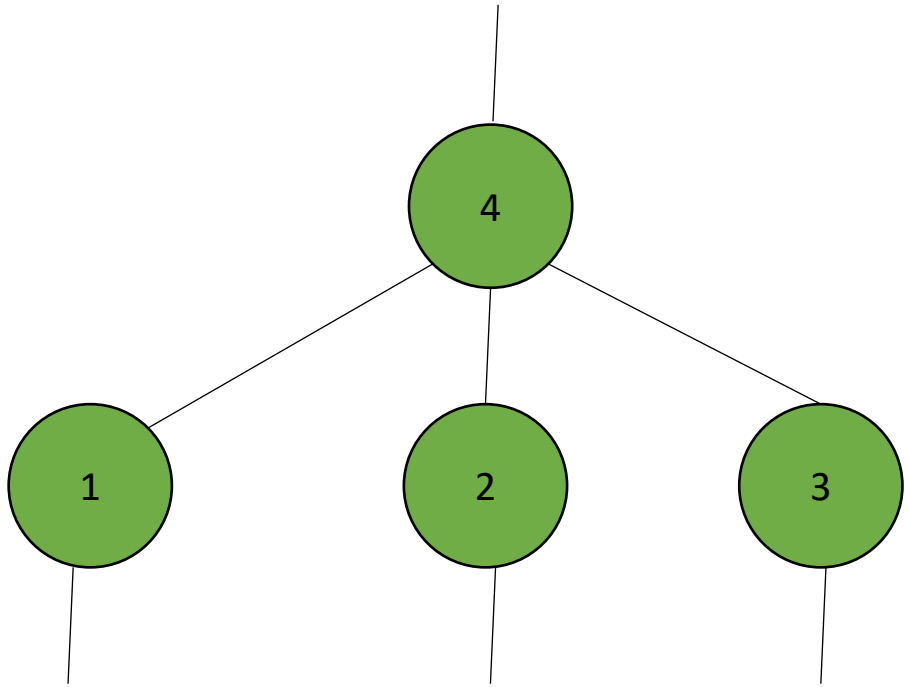
Post order traversal
For 3 children nodes

Post order traversal



Post order traversal
For 3 children nodes

Post order Traversal == Postfix Notation

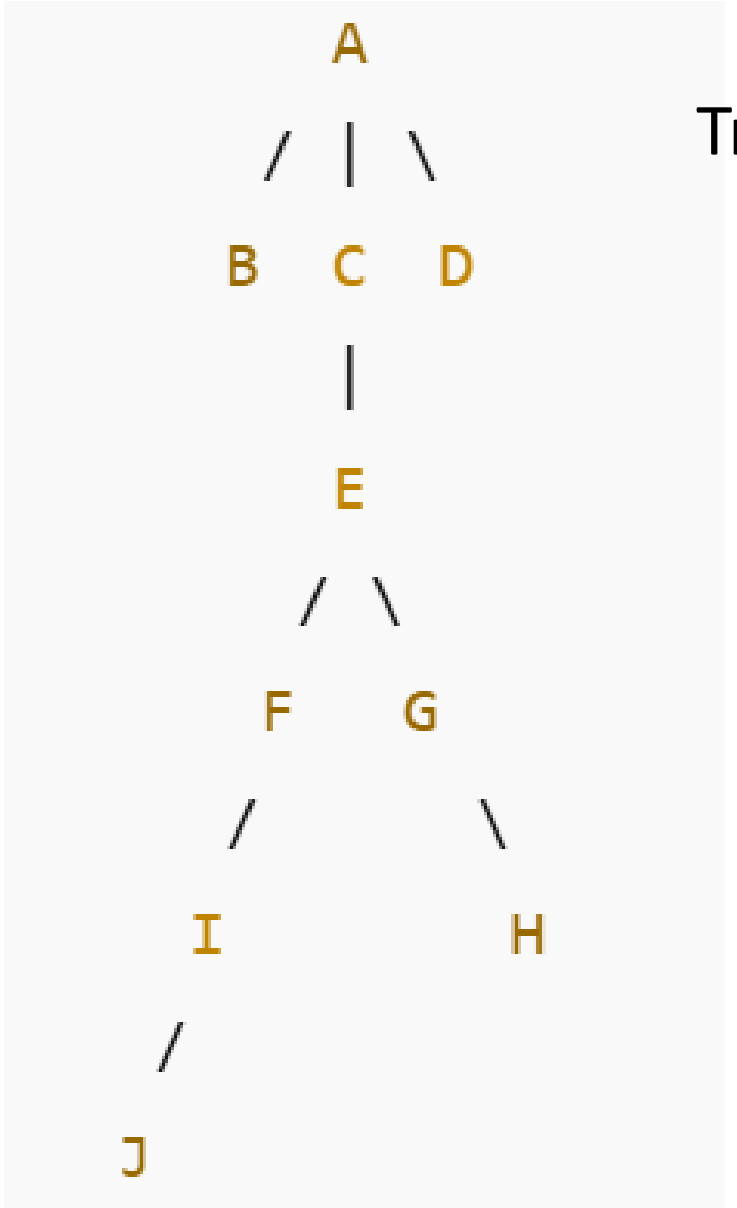


Post-order traversal of a tree ensures that all *required semantic information* for a node is available before the node is processed.

The information is stored in the node's children. For expressions this is similar to postfix notation (RPN, reverse polish notation).

Example: For an N-ary operator, evaluate all N arguments first, then invoke the operator.

Post-Order Traversal : Another Example



Traversal:

B →

J → I → F →

H → G →

E → C →

D →

A

Visit children

bottom-up

left to right,

then parent)